研究可信度驱动的期货量化系统演化式开发

课程框架、认知路径与架构护栏

摘要

对于一名从传统宏观研究转向期货量化研究的个人研究者而言，系统开发面临两类相互关联、又彼此存在张力的问题。

第一类问题是认知问题。量化系统通常被划分为数据引擎、因子框架、回测引擎、组合构建、风险管理、绩效归因和实盘执行等模块。但这种“模块清单式”的讲授方式，更接近已经熟悉量化系统的开发者视角。对于初学者而言，“Bar级最小闭环”“期货账户语义”“实验框架”“Tick撮合”等概念虽然可以分别解释，却缺少一条能够说明其出现原因、先后顺序和研究价值的认知主线。学习者容易知道系统“有哪些零件”，却不知道为什么此刻需要某个零件，缺少它会造成什么研究错误，以及增加该模块后，研究结论究竟比此前更可信在哪里。

第二类问题是工程问题。如果按照研究认知逐步深入的方法开发系统，每一次研究可信度的提升都会暴露新的市场约束和业务语义。例如，从简单收益计算进入Bar级交易模拟后，需要区分信号、目标仓位、订单、成交和实际持仓；从简化账户进入完整期货账户后，需要增加保证金、逐日盯市、今昨仓和平今手续费；从低频策略进入日内策略后，又需要更细粒度的市场事件与撮合模型。若早期代码只服务于当前最简需求，后续每一次认知升级都可能引发大规模重构，甚至推倒重来。反之，若为了避免重构而过早设计“终极系统”，则又容易陷入过度抽象和基础设施建设，迟迟不能开展真正的策略研究。

本文提出一套“双层递进”的系统开发方法。

在认知层面，以具体策略研究为主线，将系统开发组织为“研究可信度递进”的过程：

研究问题
→ 隐含假设
→ 潜在错误
→ 最小系统需求
→ 新的研究能力
→ 更强的研究结论

在工程层面，以稳定的交易概念骨架作为架构护栏，使业务语义能够逐步丰富，而基本数据流不被反复推翻：

MarketEvent
→ Strategy
→ TargetPosition
→ Order
→ Fill
→ Account
→ Performance

本文主张，第一版系统应当做到“架构上闭环，功能上极简”。它不必具备完整的期货交易语义，但应从一开始尊重几项几乎不会改变的交易事实：信号不是持仓，目标仓位不是订单，订单不是成交，只有成交能够改变实际持仓；连续合约是研究对象，物理合约才是实际交易对象；账户是资金和盈亏状态的唯一权威来源。只要

这些核心边界保持稳定，后续从Bar级回测扩展到完整期货账户、换月执行、多品种组合、实验验证和Tick撮合，就可以主要表现为局部实现替换、状态扩展和有限接口升级，而不是整个代码库的重建。

一、问题背景与总体目标

1.1 总体目标

本文讨论的目标，不是建设一个覆盖全部交易基础设施的机构级量化平台，而是：

> 开发一套支持严肃策略研究的个人期货量化研究平台。

“支持严肃策略研究”至少包含三层含义。

第一，研究对象必须被正确表示。期货市场中的交易日、夜盘、交易时段、主力合约、连续合约、换月、涨跌停、合约生命周期等制度性事实，不能被粗糙处理。

第二，策略收益必须建立在清晰的交易假设上。研究者必须知道信号何时产生、订单何时提交、按照什么价格成交、交易成本如何计入、资金和保证金如何变化。

第三，研究结论必须能够复现和验证。策略表现不能仅仅体现为一条历史收益曲线，而应能够接受样本外检验、参数敏感性分析、成本压力测试和收益归因。

在这一目标下，系统建设的主线不应是“把所有可能用到的模块一次性补齐”，而应当是：

> 先建立交易状态演化模型，再让策略研究和系统开发交替推进。

换言之，系统开发不再被视为策略研究开始前必须全部完成的准备工作，而是策略研究过程本身的一部分。每一次具体研究都会暴露新的系统需求；系统增加相应能力后，又允许研究者提出更严格、更接近真实交易的问题。

1.2 已形成的指导思想

经过前期讨论，可以将后续开发的指导思想归纳为以下几点。

第一，下一阶段不应继续无限扩张数据基础设施。数据引擎已经提供了足以支持第一轮策略研究的市场表示层，应当尽快进入交易闭环。

第二，回测系统初期不需要大而全。第一版的目的，是使“市场数据—策略判断—交易意图—成交结果—账户变化—绩效输出”完整运行，而不是立即支持所有订单类型、所有账户规则和所有市场场景。

第三，策略研究和系统演化应当交替进行。不是先建设一个完整平台，再开始策略研究；也不是用一次性脚本完成所有研究，而应采用纵向切片的方法：选择一个具体策略，打通最小闭环，再由研究中暴露的问题推动系统迭代。

第四，实盘系统可以无限期推迟。实盘接入、断线重连、多账户、容灾、实时风控和柜台同步，主要解决运行稳定性和机构运维问题，不是当前阶段理解策略和提高研究可信度的关键。

第五，应当区分“必须亲力亲为的认知工作”和“可以交给AI的工程工作”。研究者必须控制市场语义、信息时点、交易状态转换、账户不变量、组合逻辑和实验设计；AI可以大量参与样板代码、配置、日志、序列化、报告、测试框架和局部重构。

二、为什么模块清单式讲法不适合初学者

2.1 模块名称无法自动形成认知顺序

在成熟的量化系统中，开发者通常会列出如下模块：

- 数据引擎；
- 因子与信号框架；
- 组合构建；
- 回测引擎；
- 撮合执行；
- 账户与保证金；
- 风险管理；
- 绩效分析；
- 实验管理；
- 实盘执行。

这种分类本身没有错误，但它隐含了大量前置知识。

例如，对熟悉回测系统的人而言，“完善期货账户语义”意味着加入保证金、逐日盯市、今昨仓、结算和手续费规则；“接入Tick快照”意味着改变市场事件粒度和执行模型；“建设实验框架”意味着记录参数、数据版本、代码版本和样本划分。

但初学者看到的往往只是若干并列名词。他可能知道每个模块大致做什么，却难以理解：

- 为什么先做Bar而不是直接做Tick；
- 为什么账户语义不能一开始就全部实现；
- 为什么实验框架不是单纯的工程便利设施；
- 为什么组合构建不是策略完成之后的附加步骤；
- 为什么因子框架和回测引擎应该交替开发，而不是完全串行。

模块清单回答的是“系统由什么构成”，却没有充分回答“系统为什么以这个顺序形成”。

2.2 初学者真正需要的是因果链条

对于初学者，最有效的认知方式不是先理解全部系统组件，而是观察一个策略研究问题如何逐步遇到障碍。

例如，研究者从一个最简单的问题出发：

> 过去20个交易日上涨的期货品种，未来是否更容易继续上涨？

最初只需要价格和收益率。随后他会发现：

- 当日收盘后产生的信号不能按当日收盘价成交；

- 不同品种波动率不同，不能都持有一手；
- 目标仓位变化不等于实际成交；
- 手续费和滑点会侵蚀收益；
- 资金有限，保证金会约束持仓；
- 主力合约切换时，连续价格不是实际可交易资产；
- 一条漂亮的收益曲线可能只依赖少数年份或某个参数；
- 日内策略中，Bar无法判断价格路径和真实成交可能性。

这些问题一旦真实出现，相应模块的必要性就变得直观。

所以，更适合初学者的讲法是：

> 先展示研究问题，再展示当前简化假设；
> 再说明这些假设会造成什么错误；
> 最后由错误自然推出新的系统能力。

系统模块不再是预先规定的课程目录，而是研究可信度提高过程中必然出现的解决方案。

三、课程讲授的核心转换：从模块递进到研究可信度递进

3.1 课程的基本认知链条

“量化系统开发路径”课程可以使用如下固定逻辑：

> 当前想研究什么
> → 当前最简策略如何定义
> → 当前隐含了哪些假设
> → 这些假设在哪些地方可能失效
> → 失效会怎样扭曲研究结论
> → 为消除这种失真，最小需要增加什么系统能力
> → 增加后，现在能够得出多强的结论

这种讲法的核心，不是让系统越来越复杂，而是让研究结论越来越有资格被相信。

例如，最初的收益计算只能支持这样的陈述：

> 价格序列似乎存在某种趋势现象。

加入无未来信息的信号时点和下一期成交后，可以进一步说：

> 这一现象在基本可交易时序下仍然存在。

加入仓位、交易成本和账户约束后，可以说：

> 这一策略在有限资金和简化交易条件下具有历史可实现性。

加入样本外检验、参数敏感性和收益归因后，才可以说：

> 结果不太可能完全来自特定参数、样本选择或少数极端时期。

加入更细粒度的执行模拟后，才可以进一步评估：

> 对于较高频策略，收益在盘口和成交约束下是否仍有实现可能。

3.2 四条同时推进的递进轴

课程不应只沿单一“阶段”推进，而应同时观察四条递进轴。

第一条：策略定义的完整程度

最开始，策略可能只是一个指标：

> 过去20日收益为正。

随后逐步成为：

- 因子；
- 信号；
- 方向判断；
- 风险标准化目标；
- 目标仓位；
- 目标手数；
- 订单；
- 真实持仓。

这条轴解决的问题是：

> 策略到底定义到了哪一步？

许多入门回测直接从信号跳到收益率，把中间的仓位、交易和账户过程全部省略。系统开发的一个重要作用，就是把这些原本被压缩的概念重新展开。

第二条：假设的显式程度

最粗糙的回测中往往隐含：

- 信号可以按同一时点价格成交；
- 所有订单都立即全部成交；
- 没有手续费；
- 没有滑点；
- 不存在资金约束；
- 涨跌停也能交易；
- 连续合约可以直接持有；
- 换月没有成本。

随着系统演化，这些隐含假设被逐一写入明确模型。

例如：

- 信号在收盘后生成；
- 订单下一交易日开盘提交；
- 成交价为开盘价加固定滑点；
- 若涨跌停或无可成交价格，则订单延迟；
- 换月产生两笔独立交易和对应成本。

这条轴解决的问题是：

> 回测结果究竟依赖哪些假设？

第三条：市场状态模拟的完整程度

最简单的策略研究可能只保存信号和收益。随后系统逐步维护：

- 目标仓位；
- 订单；
- 成交；
- 实际持仓；
- 持仓成本；
- 现金；
- 保证金；
- 可用资金；
- 已实现盈亏；
- 未实现盈亏；
- 日终结算；
- 换月状态；
- 风险状态。

这条轴解决的问题是：

> 系统是否真的模拟了一个交易账户随时间演化的过程？

第四条：研究结论的可信程度

系统复杂度并不是目的。每增加一项能力，都应明确它提高了哪一种可信度：

- 信息时点可信度；
- 成交可实现性；
- 账户自洽性；
- 期货制度一致性；
- 多品种风险可比性；
- 样本外稳定性；
- 微观执行可信度。

这条轴回答：

> 系统增加这项功能后，我们比以前多有资格说什么？

四、以时间序列趋势策略贯穿系统开发路径

4.1 为什么选择一个主线策略

课程最好使用一个策略贯穿全程，而不是一开始并列讲解大量策略。

推荐使用一个简单的时间序列趋势策略：

> 如果某期货品种过去20个交易日收益为正，则下一期做多；如果为负，则下一期做空。

选择这一策略，不是因为它最优秀，而是因为它足够简单，可以把注意力集中在系统演化本身。同时，它又能够自然暴露：

- 信号时点；
- 仓位标准化；
- 多品种组合；
- 成交成本；
- 保证金；
- 主力换月；
- 稳健性；
- 更高频数据。

此外，可以使用三个辅助策略检查系统通用性：

- 横截面动量：暴露排序、品种池和组合约束；
- 期限结构策略：暴露多合约和期货特有的持有收益；
- 日内均值回复：暴露Bar数据和成交模型的局限。

主线策略负责形成清晰认知，辅助策略负责防止系统过度绑定单一策略。

4.2 第一阶段：价格现象是否存在

最初的研究问题是：

> 过去20日收益的符号，是否与未来收益存在稳定关系？

可以用非常简单的向量化计算：

```
signal = np.sign(close.pct_change(20))
strategy_return = signal.shift(1) * close.pct_change()
```

这一阶段的目标只是概念验证。

它需要的系统能力很少：

- 历史价格；
- 收益率计算；

- 滚动窗口；
- 信号；
- 简单收益计算；
- 基础绩效指标。

但此时不能把结果称为完整策略回测。它仍隐含：

- 仓位总是固定；
- 连续价格可以代表实际交易；
- 没有合约乘数；
- 没有保证金；
- 没有交易成本；
- 信号变化可以无摩擦完成；
- 没有资金约束。

这一阶段最多支持如下结论：

> 数据中似乎存在值得继续研究的趋势现象。

这类向量化代码可以保留为研究原型，但不应直接生长为正式回测内核。快速研究脚本允许被丢弃；系统核心不应建立在信号直接乘收益率的语义上。

4.3 第二阶段：信号是否满足可交易时序

随后需要追问：

> 这个信号在历史时点真的能够被获得吗？
> 它最早可以在哪个价格上交易？

如果20日趋势信号使用当日收盘价计算，那么它只能在收盘后才完整确定。若回测却按同一收盘价成交，就产生了未来信息。

因此需要明确：

- 市场数据时间；
- 信号计算时间；
- 决策时间；
- 订单提交时间；
- 成交时间。

第一版可以采用非常保守而清晰的假设：

> 当日收盘后计算信号，下一交易日开盘提交并成交。

此时系统需要增加一个最简时间轴和事件顺序，但仍不需要复杂订单簿。

本阶段提高的是**信息时点可信度**。完成后可以更有资格说：

> 趋势现象在不使用同一时点未来信息的条件下仍然存在。

4.4 第三阶段：从信号到目标仓位

当策略扩展到多个品种时，研究者会发现，每个品种持有一手并不代表承担相同风险。

例如：

- 不同合约乘数不同；
- 不同价格水平不同；
- 不同品种波动率不同；
- 不同保证金比例不同。

因此，方向信号必须通过仓位映射转化为风险可比的目标仓位。

最简方法可以是波动率目标：

- 估计每个品种近期波动率；
- 为每个品种分配固定风险预算；
- 根据合约乘数和价格换算目标手数；
- 对手数取整；
- 设置单品种上限。

这一阶段自然产生组合构建模块。

它不是为了架构完整，而是因为“做多”和“做空”本身不构成可比策略。真实策略至少应回答：

- 做多少；
- 承担多少风险；
- 多个品种之间如何分配资本。

本阶段完成后，研究结论从“方向预测现象”升级为：

> 该信号在风险标准化的持仓框架下能够形成组合收益。

4.5 第四阶段：从目标仓位到订单与成交

假设当前实际持有5手，而新的目标仓位是8手。目标仓位并不会自动成为实际持仓，中间至少需要：

- 计算仓位差；
- 生成买入3手的订单；
- 确定订单提交时点；
- 执行模型判断是否成交；
- 生成成交记录；
- 由成交更新持仓。

于是必须区分：

> TargetPosition
> ≠ Order

≠ Fill
≠ ActualPosition

第一版成交模型可以非常简单：

- 只支持市价单；
- 下一根Bar开盘成交；
- 全部成交；
- 固定滑点；
- 固定手续费；
- 不考虑成交量约束；
- 不保留跨日订单。

虽然功能简化，但数据流必须完整：

市场事件
→ 策略判断
→ 目标仓位
→ 订单
→ 成交
→ 实际持仓

这一阶段形成真正意义上的Bar级最小交易闭环。

其提高的是**交易过程可信度**。完成后可以说：

策略收益不是由信号直接乘市场收益得到，而是通过明确的交易假设和成交过程形成。

4.6 第五阶段：从成交到资本演化

一旦系统产生真实成交，就必须回答账户问题：

- 成交后持仓是多少；
- 持仓成本如何变化；
- 手续费从哪里扣除；
- 浮动盈亏如何计算；
- 已实现盈亏如何记录；
- 总权益如何变化；
- 资金不足时是否允许继续开仓。

第一版账户可以极简，只维护：

- 现金；
- 持仓；
- 持仓均价；
- 已实现盈亏；
- 未实现盈亏；
- 手续费；
- 总权益。

但账户应当成为资金与盈亏的唯一权威来源。策略不能自己计算最终收益，也不能直接修改账户持仓。

此时系统形成完整的资本状态链：

> Fill
> → Position
> → Cash
> → PnL
> → Equity

本阶段提高的是**账户自洽性**。完成后，策略不再只是一条收益序列，而是一个有限资金账户在历史中的状态演化。

4.7 第六阶段：完善期货账户语义

最简账户仍然没有体现中国期货交易制度的核心特征。进一步研究会暴露：

- 期货采用保证金交易；
- 盈亏通常逐日盯市；
- 可用资金会影响新订单；
- 某些交易所区分今仓和昨仓；
- 平今与平昨手续费可能不同；
- 多空持仓可以分别存在；
- 结算价与收盘价可能承担不同账户功能。

于是账户内部需要扩展为：

- 可用资金；
- 占用保证金；
- 冻结资金；
- 多头持仓；
- 空头持仓；
- 今仓；
- 昨仓；
- 已实现盈亏；
- 浮动盈亏；
- 结算盈亏；
- 手续费；
- 风险度。

回测引擎也需要增加：

- 日终结算事件；
- 保证金检查；
- 订单拒绝；
- 平今与平昨选择；
- 结算后持仓成本重置或调整。

重要的是，这一阶段不应改变前面的核心数据流。策略仍然产生目标，订单仍然代表交易意图，成交仍然是账户变化的来源。主要变化应集中在账户内部和日终调度。

本阶段提高的是**期货制度一致性**。完成后才可以更严格地说：

> 策略在有限资本、保证金和期货结算制度下具有历史可实现性。

4.8 第七阶段：连续研究对象与物理交易对象

期货研究常使用连续合约构造信号，但连续合约本身不是交易所挂牌资产。

当主力合约从旧合约切换到新合约时，策略方向可能没有变化，但系统仍然需要：

- 平掉旧主力持仓；
- 开立新主力持仓；
- 处理两个合约的价格差；
- 支付两边手续费和滑点；
- 处理不能同步成交的情况。

因此必须区分：

- 策略研究对象：例如RB连续合约；
- 实际交易对象：例如RB2510、RB2601；
- 策略换向交易；
- 机械换月交易。

这一阶段需要在目标仓位与订单之间增加合约映射和换月执行逻辑。

本阶段提高的是**研究对象与交易对象一致性**。完成后，回测不再把复权连续价格误认为实际资产，而是明确模拟物理合约的持有和迁移。

4.9 第八阶段：从单品种到多品种组合

单品种跑通后，研究者会自然扩展到多个期货品种。此时出现新的问题：

- 品种之间相关性不同；
- 同一产业链可能形成重复风险暴露；
- 多个品种同时产生信号时，总保证金可能超限；
- 某些品种流动性弱；
- 单个品种可能贡献绝大多数风险；
- 总杠杆需要限制。

于是组合模块逐步增加：

- 品种池；
- 目标风险；
- 组合波动率；
- 单品种权重上限；
- 行业集中度；
- 总保证金约束；
- 换手约束；

- 流动性过滤。

风控在这一阶段不应被理解为独立的“安全附加件”，而应视为目标仓位生成的一部分。组合构建本身就是一种风险管理。

本阶段提高的是**多品种风险可比性和资本可实施性**。

4.10 第九阶段：从收益曲线到研究证据

当系统能够生成完整收益曲线后，研究问题转变为：

> 这条曲线究竟说明了什么？

需要检查：

- 是否只在某一时期有效；
- 是否由少数品种贡献；
- 是否高度依赖某个参数；
- 是否试过大量参数后只报告最优结果；
- 成本翻倍后是否仍然有效；
- 换月规则改变后是否稳定；
- 样本外表现如何；
- 是否存在方向、波动率或期限结构暴露。

因此需要建设实验与验证框架：

- 实验配置；
- 参数记录；
- 数据版本；
- 代码版本；
- 随机种子；
- 样本内、验证集和样本外；
- Walk-forward；
- 参数敏感性；
- 成本压力测试；
- 品种归因；
- 时期归因；
- 风险因子归因。

实验框架不是为了方便批量跑参数，而是为了提高研究的证据标准。

本阶段提高的是**统计与经验研究可信度**。完成后可以更有资格判断，策略表现究竟来自稳定规律、偶然性还是模型错误。

4.11 第十阶段：何时需要Tick级模拟

Tick并不是系统“高级化”后必须自动进入的阶段。只有当策略频率和交易逻辑使Bar级假设不再可信时，Tick才成为真实需求。

例如，在日内均值回复、突破、止损或限价单策略中：

- 同一根Bar内先触及止损还是止盈无法判断；
- Bar结束后产生的信号不能按Bar内部价格成交；
- 买卖价差开始显著影响收益；
- 涨跌停盘口和空队列影响可成交性；
- 成交量和盘口数量可能限制订单；
- 同一时间戳存在多次行情更新。

此时需要将：

- Bar MarketEvent替换为Tick或快照事件；
- 简化成交模型替换为盘口感知模型；
- 立即成交扩展为部分成交、未成交和撤单；
- 粗略滑点替换为买卖价差和冲击模型。

理想情况下，策略、目标仓位、订单、成交和账户的基本语义不需要改变。变化应主要集中在市场事件和执行模型。

本阶段提高的是**高频策略的微观执行可信度**。

五、课程的多条互补讲授线索

5.1 策略需求驱动

这是课程的主线。

每一个系统模块都应回答：

> 当前具体研究遇到了什么问题？
> 为什么没有这个模块就无法继续？

例如：

- 不同品种风险不可比，推动仓位标准化；
- 目标仓位无法自动实现，推动订单与成交；
- 有成交却没有资金状态，推动账户账本；
- 连续合约不能直接交易，推动换月执行；
- 收益曲线不足以证明规律，推动实验框架；
- Bar无法描述高频成交，推动Tick模拟。

5.2 典型错误驱动

对于初学者，展示错误往往比抽象定义更有效。

错误一：当日收盘信号按当日收盘成交

引出信息时点和未来信息泄露。

错误二：把信号直接视为实际仓位

引出目标仓位、订单、成交和持仓的分离。

错误三：每个品种都持有一手

引出合约乘数、波动率和风险标准化。

错误四：连续合约就是账户持有资产

引出研究对象和物理交易对象的区别。

错误五：触及限价即默认全部成交

引出执行模型、成交量和盘口约束。

错误六：只展示一条最优参数曲线

引出样本外、多重检验和实验管理。

课程可以采用“先制造一个看似合理的错误结果，再通过系统升级纠正它”的方式，使模块价值更直观。

5.3 一笔交易的生命周期

另一条重要讲授线是围绕一笔交易的完整生命周期：

市场生成数据
→ 数据被策略观察
→ 因子被计算
→ 信号被生成
→ 目标仓位被确定
→ 订单被提交
→ 订单成交、拒绝或部分成交
→ 实际持仓变化
→ 资金和保证金变化
→ 盈亏被标记和结算
→ 结果进入绩效分析

每一步都可以追问：

- 输入是什么；
- 输出是什么；
- 状态由谁维护；
- 哪些信息当时可见；
- 哪些市场规则可能阻断下一步；
- 哪些假设会影响结果。

策略需求路径解释“为什么开发”，交易生命周期路径解释“系统如何运转”。

5.4 研究结论强度递进

课程还应明确不同系统层级能够支持什么强度的结论。

系统层级	可以支持的结论
简单向量化收益	数据中可能存在某种价格现象
明确信号时点	现象在无明显未来信息条件下仍存在
仓位与成本	现象可转化为简化条件下的净策略收益
订单、成交和账户	策略在有限资金下具有历史可实现性
期货账户与换月	结果与期货制度和物理合约交易一致
样本外与归因	结果不太可能完全来自参数和样本偶然性
Tick执行	高频策略在微观交易约束下仍可能实现

这种表述使研究者知道，系统复杂度对应的是结论责任，而不是技术炫耀。

六、认知递进为何可能导致重构危机

6.1 真正危险的不是功能简化，而是概念合并

迭代式开发一定会产生重构。问题不在于是否重构，而在于重构发生在哪里。

如果第一版只是功能简单，例如：

- 只支持市价单；
- 只支持全部成交；
- 不支持今昨仓；
- 不支持部分成交；

后续通常可以通过局部扩展完成。

但如果第一版把不同概念合并，后续就会发生核心语义推翻。

最典型的错误是：

> Signal = Position = Fill

例如直接计算：

```
position = signal
strategy_return = position.shift(1) * market_return
```

这在概念验证阶段可以使用，但不能作为正式回测系统内核。因为它把以下过程全部压缩掉了：

- 策略希望持有什么；
- 当前实际持有什么；
- 需要下什么订单；
- 订单是否成交；
- 最终持仓是多少。

一旦加入成交失败、部分成交、资金不足或涨跌停，这种模型就必须整体重写。

6.2 其他容易造成全面推翻的设计

策略直接修改账户

如果策略执行：

```
account.position = target
```

那么成交模型无法插入，部分成交和拒单也无法表达。

策略自己计算最终收益

这样每个策略都会隐含一套不同的手续费、成交和账户规则，系统无法统一升级。

连续合约直接进入账户

当主力切换时，系统无法解释账户究竟持有什么资产。

所有逻辑写在一个主循环

随着功能增加，数据、信号、订单、成交、账户、结算和绩效会相互缠绕，任何修改都会波及全局。

使用一个模糊时间字段

如果市场时间、决策时间、提交时间和成交时间都被压缩成一个date或timestamp，未来信息和事件排序问题将难以修复。

因此，要避免的是“错误的概念边界”，而不是一切早期简化。

七、演化式架构的基本原则

7.1 架构上闭环，功能上极简

第一版回测系统应当具备完整交易链条：

MarketEvent
→ Strategy

→ TargetPosition
→ Order
→ Fill
→ Account
→ Performance

但每一环都可以极其简单。

例如：

- Strategy只支持一种趋势信号；
- TargetPosition只支持单品种目标手数；
- Order只支持市价单；
- Execution假设下一Bar开盘全部成交；
- Account只维护基本现金、持仓和盈亏；
- Performance只输出收益、波动率和回撤。

这种设计保留了正确的概念边界，同时避免过度开发。

7.2 稳定概念骨架，逐渐丰富业务语义

系统中有一些概念，从第一版起就应保持独立：

- 市场事件；
- 策略判断；
- 目标仓位；
- 订单；
- 成交；
- 实际持仓；
- 账户；
- 结算；
- 绩效快照。

随着认知深化，可以逐步增加字段和状态。

例如，账户可以从：

- cash；
- position；
- equity；

扩展到：

- available_cash；
- margin；
- frozen_margin；
- long_position；
- short_position；
- today_position；
- yesterday_position；
- realized_pnl；

- unrealized_pnl；
- settlement_pnl。

只要账户对外仍保持类似接口：

```
account.apply_fill(fill)
account.mark_to_market(market)
account.settle(trading_date)
account.snapshot()
```

内部丰富不会迫使策略和信号模块全面重写。

7.3 提前分离职责，延迟复杂抽象

应当从第一版就分离：

- 策略与执行；
- 执行与账户；
- 研究对象与交易对象；
- 手续费与账户；
- 数据输入与策略逻辑。

但不应过早建立复杂继承体系、插件系统或微服务。

例如，第一版只有一种手续费规则时，不需要设计十几层手续费类；只需要保证手续费计算不被硬编码在策略或主循环中。

好的抽象通常来自多个真实实现的共同部分，而不是凭空想象未来需求。

7.4 为下一阶段留下替换点，不实现遥远需求

第一版已知未来会增加：

- 更复杂账户；
- 涨跌停；
- 换月；
- 部分成交；
- Tick事件。

因此可以预留：

- ExecutionModel；
- CommissionModel；
- SlippageModel；
- Account接口；
- ContractResolver；
- MarketEvent抽象。

但不需要提前实现所有复杂逻辑。

原则是：

> 为明确可见的下一阶段留下接口，
> 不为遥远且不确定的可能性建设功能。

八、第一版必须守住的架构不变量

以下不变量应当被视为系统最重要的护栏。

8.1 策略不能直接修改实际持仓

策略只能表达：

- 信号；
- 目标风险；
- 目标仓位；
- 交易意图。

实际持仓必须由成交决定。

8.2 实际持仓只能由成交事件改变

无成交，不改变持仓。即使目标仓位变化，也必须通过订单和成交实现。

8.3 信号、目标仓位、订单、成交和持仓相互分离

它们分别代表：

- 对未来的判断；
- 希望承担的风险；
- 提交给市场的意图；
- 市场确认的结果；
- 当前账户事实。

8.4 连续合约是研究对象，物理合约是交易对象

研究策略可以针对连续序列，但账户必须持有真实挂牌合约。

8.5 账户是资金和盈亏状态的唯一权威来源

策略、绩效和执行模块不能各自维护一套账户结果。

8.6 手续费、滑点和执行模型必须可替换

策略不应知道自己使用固定滑点、对手价还是盘口撮合。

8.7 市场时间、决策时间、订单时间和成交时间必须明确

它们可以在简化模型中相同或相邻，但语义不能混淆。

8.8 回测结果必须由系统状态产生，而不是由策略直接计算

绩效模块应读取账户快照、成交记录和持仓历史，而不是让策略自己返回“收益”。

九、从Bar级闭环到完整期货账户的演化示例

9.1 V1：最小账户

第一版账户可以只包含：

```
class Account:
    cash: float
    positions: dict[str, int]
    equity: float

    def apply_fill(self, fill): ...
    def mark_to_market(self, market): ...
    def snapshot(self): ...
```

此时系统假设：

- 单一持仓方向；
- 不区分今昨仓；
- 保证金简化或暂不单独维护；
- 每笔成交立即更新资金；
- 不考虑复杂结算。

9.2 V2：期货账户

后续扩展为：

```
class FuturesAccount:
    cash: float
    available_cash: float
    margin: float
    realized_pnl: float
    unrealized_pnl: float
    positions: PositionBook

    def apply_fill(self, fill): ...
    def mark_to_market(self, market): ...
    def can_accept(self, order, market): ...
```

```
    def settle(self, settlement_prices): ...
    def snapshot(self): ...
```

同时增加：

- PositionBook；
- 今仓与昨仓；
- 多头与空头；
- 保证金；
- 平今与平昨；
- 日终结算；
- 资金不足拒单。

9.3 哪些部分保持不变

理想情况下：

- Strategy不需要重写；
- Signal不需要重写；
- TargetPosition不需要重写；
- Order基本不变；
- Fill基本不变；
- Execution仍然输出Fill；
- Performance仍然读取账户快照。

主要变化集中在：

- Account内部；
- PositionBook；
- MarginModel；
- CommissionModel；
- BacktestEngine的日终调度；
- 订单接受检查。

这属于健康演化。

9.4 哪些接口可能合理扩展

例如，执行接口最初可能是：

```
execute(order, bar) -> Fill
```

后来变为：

```
execute(order, market_state, account_state) -> list[Fill]
```

这会修改部分调用方，但没有改变系统方向，属于可接受的接口升级。

真正危险的是从“策略直接返回持仓和收益”改成“策略返回目标、订单成交后账户计算收益”。这属于核心语义推翻。

十、测试如何使重构可控

10.1 测试不是附属工程，而是架构保护机制

没有测试时，AI Agent或人工重构很容易制造“能够运行，但业务语义已经改变”的系统。

测试需要保护的不是某个函数当前写法，而是系统不变量。

10.2 模块不变量测试

订单不变量

- 成交数量不能超过订单数量；
- 已撤销订单不能继续成交；
- 被拒订单不能改变持仓；
- 成交时间不能早于订单时间。

持仓不变量

- 新持仓等于旧持仓加带方向的成交量；
- 无成交时持仓不变；
- 平仓量不能超过可平仓持仓。

账户不变量

- 账户权益计算必须自洽；
- 手续费必须减少权益；
- 保证金占用不能凭空消失；
- 结算前后资金变化必须符合定义。

时间不变量

- 策略不能读取决策时点之后的数据；
- 成交不能发生在订单提交之前；
- 账户快照不能包含未来成交。

10.3 黄金路径测试

应构造一个小型、可手工计算的完整场景，例如：

- 初始资金100万元；
- 第一天无仓；
- 第二天买入2手；
- 第三天价格上涨；
- 第四天卖出平仓；
- 计算手续费、持仓盈亏、可用资金和总权益。

每一步预期结果都应人工确定。

以后无论如何重构，黄金路径结果必须保持一致。

10.4 特殊期货场景测试

逐步增加：

- 涨停无法买入；
- 跌停无法卖出；
- 平今与平昨手续费不同；
- 资金不足拒单；
- 主力换月平旧开新；
- 夜盘跨自然日但属于同一交易日；
- 日终结算后持仓成本调整；
- 同一时间戳多个合约事件的顺序。

测试越接近业务不变量，系统越能安全演化。

十一、AI Agent在演化式开发中的角色

11.1 适合AI完成的工作

AI适合：

- 数据类和接口样板；
- 配置解析；
- 日志；
- 序列化；
- CLI；
- 报告；
- Mock对象；
- 单元测试初稿；
- 参数遍历；
- 局部性能优化；
- 类型注解；
- 文档；
- 有明确测试保护的局部重构。

11.2 必须由研究者控制的工作

研究者必须亲自定义：

- 信号时点；
- 事件顺序；
- 订单和成交语义；
- 账户状态转换；
- 保证金和结算规则；

- 连续合约与物理合约的关系；
- 风险预算；
- 实验设计；
- 系统不变量。

11.3 AI参与重构的前提

AI能够有效降低重构成本，但前提是：

- 模块职责清晰；
- 接口契约明确；
- 不变量已经写成测试；
- 黄金路径可回归；
- 变更范围被限定。

否则，AI只会更快地生成大量表面一致、内部语义漂移的代码。

十二、面向后续开发的综合实施框架

12.1 每个阶段固定回答十个问题

后续课程或实际开发中，每一阶段都可以使用以下模板：

1. 当前研究问题是什么；
2. 当前最简策略如何定义；
3. 数据在什么时点可得；
4. 当前隐含了哪些交易假设；
5. 最可能出现什么研究错误；
6. 当前结论最多能支持什么表述；
7. 为消除关键错误，需要增加什么最小系统能力；
8. 哪些系统契约必须保持不变；
9. 哪些实现允许替换或扩展；
10. 本阶段完成的验收标准是什么。

12.2 推荐版本路线

V0：研究原型

- 向量化验证；
- 快速尝试因子；
- 允许丢弃；
- 不作为正式回测内核。

V1：最小状态闭环

- Bar数据；
- 单策略；
- TargetPosition；
- Order；

- Fill；
- 简单Account；
- 固定手续费和滑点；
- 基础绩效。

V2：期货账户语义

- 保证金；
- 逐日盯市；
- 今昨仓；
- 平今手续费；
- 资金检查；
- 日终结算。

V3：物理执行语义

- 主力换月；
- 物理合约映射；
- 涨跌停；
- 未成交订单；
- 部分成交；
- 流动性约束。

V4：组合与实验语义

- 多品种；
- 风险预算；
- 组合约束；
- 样本划分；
- 参数敏感性；
- 成本压力测试；
- 归因；
- 实验版本。

V5：更细市场事件

- Tick或快照；
- 买卖价差；
- 盘口；
- 订单生命周期；
- 更细撮合。

12.3 健康重构与危险重构对照

类型	示例	判断
局部实现重构	固定手续费改为分品种手续费	健康
状态扩展	Position增加今仓和昨仓	健康
接口扩展	execute增加account_state参数	可接受
模型替换	BarExecution替换为TickExecution	健康

类型	示例	判断
核心语义推翻	策略直接持仓改为订单成交后持仓	高成本
对象身份推翻	连续合约账户改为物理合约账户	高成本
权威来源推翻	策略计算收益改为账户计算收益	高成本

12.4 第一版可演化性检查表

在正式编码V1前，应检查：

- 策略是否只表达判断和目标；
- 实际持仓是否只能由成交改变；
- 订单和成交是否独立；
- 连续合约和物理合约是否可区分；
- 账户是否是资金与盈亏的唯一来源；
- 手续费和滑点是否可替换；
- 时间语义是否明确；
- Bar改成Tick时，策略和账户能否原则上复用；
- 增加账户字段时，是否主要修改账户内部；
- 是否存在至少一条完整黄金路径测试。

十三、结论

个人期货量化研究平台的开发，不应被理解为按照固定模块清单逐项施工，也不应被理解为先完成全部基础设施，再开始策略研究。

更合理的路线是：

> 以具体策略研究为认知主线，
> 以研究结论可信度为递进标准，
> 以稳定交易概念为架构护栏，
> 以局部可替换实现支持系统演化。

在认知层面，课程应从“系统有哪些模块”转向：

> 当前研究想回答什么问题；
> 当前结论依赖哪些假设；
> 哪些错误会使结论失真；
> 为消除这些错误，需要增加什么最小能力。

在工程层面，第一版不必功能完整，但必须守住核心概念边界：

> 信号不是持仓；
> 目标仓位不是订单；
> 订单不是成交；
> 只有成交改变实际持仓；

连续合约不是实际交易资产；
账户是资金与盈亏的唯一权威来源。

系统开发的理想状态不是“从不重构”，而是：

- 核心概念和数据流保持稳定；
- 业务语义持续丰富；
- 旧实现被局部替换；
- 接口只做有限扩展；
- 测试确保系统语义不漂移。

因此，量化系统演化可以概括为一套双层结构：

上层：研究可信度不断提高；
下层：稳定架构骨架上的业务语义不断丰富。

策略研究决定系统下一步为什么演化，架构护栏决定系统怎样演化而不推倒重来。二者结合，才能避免两个极端：一方面，不会陷入无边界的基础设施建设；另一方面，也不会因为一次性脚本缺乏概念边界，而在每次认知升级时重新建设整套系统。

最终，这套方法的目标并不是打造一套技术上最复杂的量化平台，而是建设一个能够持续支持研究者提出更严格问题、发现更隐蔽错误、形成更可信结论的个人研究环境。